KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

Project Report

SECURE FILE STORAGE POOL

Submitted to Bharathiar University in partial fulfillment of the requirements for the award of the degree of

Bachelor of Computer Applications

Submitted by
VEERAANJALI M
(2322J0369)

Under the Supervision and Guidance of

Mrs.R.Vinodhini,M.Sc.,M.Phil. ,(Ph.D)

Assistant Professor

Department of Computer Applications

KG COLLEGE OF ARTS AND SCIENCE

March 2026

DECLARATION

DECLARATION

I declare that the work which is being presented in this project report entitled **SECURE FILE STORAGE POOL**, submitted to the Department of Computer Applications, KG College of Arts and Science, Saravanampatti, Coimbatore for the award of the degree of Bachelor of Computer Applications of the Bharathiar University, is an authentic record of my work carried out under the supervision and guidance of **Mrs.R.Vinodhini,M.Sc.,M.Phil.,(Ph.D).,** I have not plagiarized or submitted the same work for the award of any other degree.

March 2026

Coimbatore

(VEERAANJALI M)

CERTIFICATE

CERTIFICATE

This is to certify that the project entitled **SECURE FILE STORAGE POOL** submitted to Bharathiar University in partial fulfillment for the award of the degree of Bachelor of Computer Applications is a record of original work done by **VEERAANJALI M (2322J0369)** during the period of study in **KG College of Arts and Science** under the supervision of **Mrs.R.Vinodhini,M.Sc.,M.Phil. ,(Ph.D).**, Department of Computer Applications.

Place:Coimbatore

Date:

Signature of the Guide Signature of the HoD

(College Seal)

Submitted for the Viva-Voce Examination held on

Internal Examiner **External Examiner**

g-net solutions®

New Technologies for business growth

No: 303-B, PSG Software Park II,

Peelamedu, Coimbatore- 641 004.

Website: www.g-netsolutions.com

Phone: +91-0422-259 5080.

MARCH 2026

PLACE : Coimbatore

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **VEERAANJALI M (REGNO: 2322J0369)** studying in the department of III B.C.A a student of **KG COLLEGE OF ARTS AND SCIENCE, COIMBATORE** had completed her internship under the title **" SECURE FILE STORAGE POOL "** in our company from **15.12.2025 to 28.02.2026.**

For g-net solutions Coimbatore Private Limited,

G-NET SOLUTIONS P. LTD. COIMBATORE CBE-4

HR Signature

ACKNOWLEDEMENT

ACKNOWLEDGEMENT

I express my sincere thanks to **Dr. Ashok Bakthavathsalam, B.E, MS., Ph.D**. Managing Trustee of KG College of Arts and Science for giving me an opportunity to do this course of study and to undertake this project work.

It is my pleasure to express my gratitude to **Dr. B. Vanitha, M.A., M.Phil., Ph.D.,** Secretary, KG College of Arts and Science, for her valuable thoughts and guidance for completing this project successfully.

I express my humble thanks to **Dr. S.Vidhya, MCA.,M.Phil., Ph.D.,** NET, Principal, KG College of Arts and Science, for encouraging and helping me for the completion of this project.

I express my sincere thanks to **Dr. M.Usha, M.Sc,M.Phil.,Ph.D.,** Dean-School of Computational Science, KG College of Arts and Science, for her constant motivation and endless support.

I have great pleasure in acknowledging my thanks to **Dr.P.Vijayakumar, M.Sc., M.Phil., Ph.D.,** Associate professor Head, Department of Computer Applications, KG College of Arts and Science for his encouragement and help throughout the course.

I express my profound thanks to my guide **Mrs. R. Vinodhini, M.Sc.,M.Phil.,(Ph.D)** Assistant Professor, Department of Computer Applications, KG College of Arts and Science, for her constant encouragement and motivation throughout the project.

- VEERAANJALI M

CONTENTS

CONTENTS

S.NO	PARTICULARS	PAGE NO
	ACKNOWLEDGEMENT	
	CONTENTS	
	SYNOPSIS	
1	**INTRODUCTION**	**1**
	1.1 ORGANIZATION PROFILE	**2**
	1.2 SYSTEM SPECIFICATION	**3**
	1.2.1 HARDWARE REQUIREMENTS	3
	1.2.2 SOFTWARE REQUIREMENTS	3
2	**SYSTEM STUDY**	**4**
	2.1 EXISTING SYSTEM	4
	2.1.1 DRAWBACKS	4
	2.2 PROPOSED SYSTEM	**5**
	2.2.1 FEATURES	5
3	**SYSTEM DESIGN AND DEVELOPMENT**	**6**
	3.1 FILE DESIGN	**6**
	3.2 INPUT DESIGN	**7**
	3.3 OUTPUT DESIGN	**7**
	3.4 DATABASE DESIGN	**8**
	3.5 SYSTEM DEVELOPMENT	**9**
	3.5.1 DESCRIPTION OF MODULES	9
4	**TESTING AND IMPLEMENTATION**	**12**
	4.1 TESTING	12
5	**CONCLUSION**	**14**
	BIBLIOGRAPHY	**15**
	APPENDICES	**16**

	A. DATA FLOW DIAGRAM	16
	B. TABLE STRUCTURE	**18**
	C. SAMPLE CODING	**20**
	D. SAMPLE INPUT	**28**
	E. SAMPLE OUTPUT	**31**

SYNOPSIS

SYNOPSIS

The Secure File Storage Portal is a web-based application designed to provide a secure environment for storing, sharing, and managing files. It employs client-side encryption to ensure that files are encrypted before being uploaded to cloud storage, protecting sensitive data from unauthorized access. Users register and log in to access a personal dashboard where they can upload files, enter a secret passphrase to encrypt them using AES-256-GCM encryption, and securely store both the encrypted data and its associated metadata. The system utilizes Firebase Hosting for serving the application and Firebase Fire store for storing encrypted files and metadata. Downloading files requires the correct passphrase to decrypt the data entirely on the client side, ensuring a zero-knowledge security model where neither the server nor Firebase has access to the unencrypted files or passphrases. This project focuses on data confidentiality, integrity, and user control over sensitive files in a serverless, scalable environment.

The project integrates modern web cryptography standards to derive strong encryption keys from user passphrases using PBKDF2 and applies authenticated encryption to protect files against tampering and unauthorized decryption. By performing all encryption and decryption in the user's browser, the system eliminates the risk of exposing sensitive data during transfer or storage. User authentication is managed locally for demonstration purposes but can be extended to use Firebase Authentication for enhanced security and scalability.

INTRODUCTION

1. INTRODUCTION

In today's digital world, the need to securely store and share sensitive information has become critical. Traditional cloud storage services often store files in unencrypted form or rely on server side encryption, which can expose data to unauthorized access or breaches. The Secure File Storage Portal addresses these challenges by implementing client-side encryption, ensuring that files are encrypted before they leave the user's device, thus maintaining confidentiality throughout storage and transmission. This project provides a web-based platform where users can register, log in, and securely manage their files with strong encryption mechanisms. Using AES-256-GCM encryption combined with PBKDF2 key derivation, the system ensures data integrity and protection against tampering. By leveraging Firebase Hosting and Fire store, the application offers a scalable and serverless architecture, enabling users to upload, store, retrieve, and decrypt files in a seamless and secure manner. This approach empowers users with complete control over their data privacy, following a zero-knowledge security model where the server never accesses plaintext files or encryption keys.

The implementation of client-side cryptography eliminates the risk of data exposure during transfer or storage on third-party servers. Users' passphrases are never transmitted or stored, ensuring that even if the backend storage is compromised, the encrypted files remain secure and inaccessible without the correct key. Additionally, the use of modern web cryptographic APIs allows the system to run efficiently in standard browsers without requiring additional software installations, providing a user-friendly experience. Moreover, the project emphasizes ease of use and accessibility through its simple and intuitive interface. Users can effortlessly upload files, provide passphrases for encryption, view their stored encrypted files, and securely download and decrypt them whenever required. This design makes the Secure File Storage Portal a practical solution for individuals and organizations seeking robust data protection without sacrificing usability or performance.

1.1 ORGANIZATION PROFILE

G-Net Solutions is a technology-driven organization dedicated to creating secure, innovative web applications that address critical data privacy challenges. Located in Peelamedu, Coimbatore, the company specializes in developing scalable and user-centric digital solutions by integrating advanced client-side cryptography with modern cloud services. For this project, G Net Solutions has leveraged expertise in web security, client-side encryption, and serverless cloud deployment to deliver a secure file storage platform that prioritizes user data confidentiality.

The organization focuses on building applications that ensure robust data protection while maintaining simplicity and accessibility. By utilizing the Web Crypto API for client-side encryption and Firebase Hosting combined with Fire store for cloud storage, G-Net Solutions delivers a lightweight and efficient solution that eliminates the need for server-side decryption or complex backend infrastructure. This approach caters to users ranging from individuals to organizations seeking secure file management without compromising usability.

Committed to high standards in software development, G-Net Solutions emphasizes security, maintainability, and performance. The Secure File Storage Portal exemplifies the company's commitment to empowering users with full control over their sensitive data through zero knowledge encryption models and serverless architectures. Through projects like this, G-Net Solutions continues to innovate in cybersecurity solutions that protect data privacy in an increasingly connected digital world.

SYSTEM SPECIFICATION

1.1.1 HARDWARE REQUIREMENTS

1. Processor: Intel i3 or above
2. RAM: 4GB or above
3. Hard Disk: 20GB free space

1.1.2 SOFTWARE REQUIREMENTS

1. Frontend: HTML, CSS, JavaScript
2. Backend: Spring Boot (Java)
3. Database: MySQL
4. Hosting: Firebase
5. IDE: VS Code / IntelliJ

NON-FUNCTIONAL REQUIREMENTS

1. Security: AES Encryption
2. Performance: Fast file upload/download
3. Scalability: Supports multiple users
4. Reliability: Secure data storage

SYSTEM STUDY

2 SYSTEM STUDY

2.1 EXISTING SYSTEM

File storage solutions primarily rely on server-side encryption or simple cloud storage services that store files in plaintext or use encryption only during transit. Many popular cloud platforms offer convenient file sharing and backup but do not guarantee complete data privacy because encryption keys are often managed by the service provider. Users must trust these providers to secure their files, which exposes sensitive data to potential unauthorized access or breaches.

The most existing systems perform encryption and decryption on the server, increasing the risk of data exposure during processing. Users have limited control over encryption keys, and data breaches can compromise unencrypted information. The lack of client-side encryption forces users to depend on backend security measures and reduces transparency regarding data confidentiality.

DRAWBACKS

- ❖ Reliance on server-side encryption increases the risk of data exposure during file handling.
- ❖ Encryption keys are often stored or managed by cloud providers, reducing user control and data privacy.
- ❖ Potential for unauthorized access due to centralized key management and server vulnerabilities.
- ❖ Limited transparency in how data is encrypted and handled, reducing user trust.
- ❖ Lack of client-side encryption leads to exposure of files in plaintext on the server or during transit

2.2 PROPOSED SYSTEM

The proposed Secure File Storage Portal is a web-based platform that ensures robust security by performing all encryption and decryption on the client side before any data is uploaded to cloud storage. Users encrypt files locally using AES-256-GCM encryption derived from their passphrases, ensuring that only encrypted data is stored remotely. The system uses Firebase Hosting for deployment and Fire store for storing encrypted file metadata while keeping users' encryption keys private and never transmitted or stored.

This zero-knowledge approach guarantees that neither the server nor any third party can access the unencrypted files or keys, offering complete control and privacy to the user. The portal provides an intuitive interface for user registration, login, secure file upload, encrypted file listing, and safe download with decryption performed entirely in the browser. This client-side encryption mechanism safeguards sensitive data even if cloud storage is compromised.

2.2.1 FEATURES

- ❖ Client-side AES-256-GCM encryption and decryption using passphrase derived keys to protect files.
- ❖ Zero-knowledge architecture where encryption keys and plaintext files never leave the user's device.
- ❖ User registration and authentication to maintain personalized secure file storage.
- ❖ Secure upload of encrypted files along with metadata (IV, salt) stored in Firebase Fire store.
- ❖ Encrypted file download with in-browser decryption requiring the correct passphrase.
- ❖ Intuitive dashboard for uploading, viewing, downloading, and deleting encrypted files.
- ❖ Serverless deployment using Firebase Hosting and Fire store, ensuring scalability and ease of maintenance.
- ❖ Cross-platform accessibility via standard modern web browsers with no additional software needed.

SYSTEM DESIGN AND DEVELOPMENT

3 SYSTEM DESIGN AND DEVELOPMENT

The system design and development phase for the Secure File Storage Portal focuses on creating a secure, web-based platform that enables users to encrypt, upload, store, and decrypt files entirely on the client side before interacting with cloud storage. The design prioritizes strong client-side encryption using the Web Crypto API, zero-knowledge architecture where encryption keys never leave the user's device, and a simple yet secure user interface for seamless file management.

The portal is implemented as a modular single-page application (SPA) using modern JavaScript with Vite for fast builds and optimized performance. Core functionalities such as user authentication, client-side file encryption and decryption, secure upload and download, and file metadata management are handled in-browser, minimizing reliance on backend processing and enhancing data privacy. Integration with Firebase Hosting and Firestore provides secure cloud storage for encrypted files and metadata, while client-side cryptographic operations protect sensitive data from unauthorized access.

3.1 FILE DESIGN

The file structure of the Secure File Storage Portal is organized to support modularity, maintainability, and clear separation of concerns. The project is divided into key modules including:

- **Authentication Module:** Manages user registration, login, and session handling.
- **Encryption Module:** Contains cryptographic utilities for AES-GCM encryption and decryption using passphrase-derived keys.
- **Storage Module:** Handles interaction with Indexed DB (or optionally Firebase Fire store) to save encrypted file blobs and metadata securely.

- **UI Components:** Separate scripts or components for handling file input, file listing, download/decrypt operations, and user notifications.

- **Configuration Files:** Manage project dependencies, build configurations, and deployment settings using Vite and npm.

This modular structure enables independent development, easier debugging, and future scalability without impacting unrelated parts of the system.

3.2 INPUT DESIGN

The system's input design emphasizes usability and security for managing encrypted file storage. Key user inputs include:

- **File Type**: Users select file type such as text, image, audio, pdf, video. The system validates what the file type was.
- **File Upload:** Users select files from their local device via a file input control. The system validates file presence before proceeding.
- **Encryption Passphrase:** Users provide a passphrase used to derive encryption keys. This passphrase is never stored or transmitted, ensuring zero-knowledge security.
- **Authentication Credentials:** Email and password inputs for user registration and login to control access to personal file storage.
- **Action Buttons:** Clear controls to initiate file upload (encryption and storage), download (decryption), and logout.

Input validations and prompt feedback ensure smooth and secure interaction from file selection to encrypted storage.

3.3 OUTPUT DESIGN

Outputs are designed to provide clear, actionable feedback to users managing encrypted files. After interactions, the system displays:

- **File List:** Displays user-specific encrypted files with metadata such as file name and upload date.
- **Raw:** Allows users to download the encrypted data or files.
- **Decrypt Button:** Allow users to enter the passphrase and download the encrypted data into decrypted data.
- **Delete Button:** Enables secure removal of stored encrypted files from the database.
- **Authentication Status:** Displays user login/logout state with appropriate UI changes.
- **Alerts & Notifications:** Inform users of successful uploads, errors (e.g., wrong passphrase), and session changes.

The interface is designed to be responsive and user-friendly, ensuring accessibility across desktop and mobile browsers.

3.4 DATABASE DESIGN

The Secure File Storage Portal uses a hybrid storage approach:

- **Client-Side Storage:** Indexed DB is used to store encrypted file blobs and associated metadata locally for offline access and quick retrieval.
- **Cloud Storage:** Firebase Fire store (or optionally Firebase Storage) stores encrypted file metadata and blobs securely in the cloud, indexed by user ID to maintain data segregation and access control.
- **Security Rules:** Firebase Security Rules enforce that only authenticated users can access their own files, ensuring privacy and data protection.

This design balances performance, security, and scalability while preserving user control over encryption keys and sensitive data.

SYSTEM DEVELOPMENT

The system is developed using modern JavaScript with Vite for rapid development and optimized builds. The Web Crypto API is employed for robust AES-GCM client-side encryption and decryption, ensuring that sensitive files are never exposed in plaintext outside the user's device. User authentication is simulated locally or integrated with Firebase Authentication for real-world deployments. The UI uses standard HTML5, CSS3, and JavaScript for a clean, minimalistic, and responsive interface supporting file upload, passphrase entry, and file management. All cryptographic operations occur within the browser environment, preventing any plaintext file or key leakage. Deployment is handled through Firebase Hosting, providing secure HTTPS access and global CDN support.

3.5 DESCRIPTION OF MODULES

Authentication Module

This module manages user registration, login, and session handling. It verifies user credentials stored securely (locally or via Firebase Auth), manages session persistence, and restricts access to the dashboard and file storage features to authenticated users only. This module ensures that only authorized users can upload, view, and manage their encrypted files.

File Upload and Validation Module

This module allows users to select files from their local device for secure storage. It performs validation to ensure a file is selected before proceeding. Upon file selection, the module triggers client-side encryption using the user-provided passphrase. This step guarantees that only encrypted data leaves the user's device, maintaining data confidentiality.

Encryption and Decryption Module

At the core of data security, this module handles all cryptographic operations using the Web Crypto API. It derives encryption keys from user passphrases via PBKDF2, generates initialization vectors (IV), and performs AES-GCM encryption on files before upload. Likewise, it decrypts encrypted blobs on download using the correct passphrase. This module enforces the zero-knowledge principle by never transmitting or storing plaintext data or encryption keys externally.

Storage Management Module

This module interfaces with Indexed DB (or optionally Firebase Fire store) to store encrypted files and their metadata securely. It handles saving encrypted blobs along with associated metadata such as file names, IVs, salts, timestamps, and user ownership information. It also supports retrieval, listing, and deletion of user-specific encrypted files, providing seamless file management.

File Listing and Interaction Module

This module generates and updates the dynamic list of encrypted files owned by the logged-in user. It provides user-friendly controls to download (with decryption), delete files, and view relevant metadata. It handles user input for decryption passphrases during download, validates inputs, and manages error feedback if decryption fails.

User Interface and State Management Module

Responsible for overall UI rendering and responsiveness, this module manages the display of login/authentication forms and the secure file dashboard. Using JavaScript state

management, it ensures smooth transitions between authentication states and file management views. It also manages user notifications, alerts, and input validations to enhance usability and security awareness.

Hosting and Deployment Module

This module covers deployment of the entire web application on Firebase Hosting. It leverages Firebase's global CDN for fast, secure content delivery over HTTPS, ensuring availability and scalability. The deployment pipeline integrates build tools like Vite for optimized production builds, enabling continuous updates and reliable hosting without server-side file processing.

TESTING AND IMPLEMENTATION

TESTING AND IMPLEMENTATION

4.1 TESTING

Testing is the process of executing the Secure File Storage Portal to identify functional errors, security flaws, and performance issues before deployment. Effective testing ensures that files are correctly encrypted on the client side, securely stored, and successfully decrypted only with the correct passphrase. The testing phase validates that the system maintains data confidentiality, integrity, and reliable user interaction.

The testing process systematically verifies each module—from authentication and encryption logic to file storage, retrieval, and deletion—ensuring the system operates securely and efficiently. Thorough testing helps detect vulnerabilities, prevents data loss, and guarantees a smooth and trustworthy user experience.

Types of Testing

- ❖ Unit Testing
- ❖ Integration Testing
- ❖ Validation Testing
- ❖ Output Testing

Unit Testing

Unit testing focuses on testing individual components of the Secure File Storage Portal independently. This includes testing the authentication logic, encryption and decryption functions, file upload handling, and Indexed DB/Firebase storage operations. Each cryptographic function is tested to ensure correct key derivation, encryption output, and successful decryption with valid passphrases. This process helps identify errors in isolated modules early, improving overall code reliability and security.

Integration Testing

Integration testing verifies the interaction between multiple modules to ensure seamless workflow execution. This includes testing the flow from user login to file upload, encryption, storage, retrieval, decryption, and download. It confirms that encrypted files are correctly passed between the encryption module and storage module, metadata is properly associated with user accounts, and decrypted files are restored accurately. These tests ensure all system components function together without conflicts.

Validation Testing

Validation testing ensures that the system meets all functional and security requirements defined for the project. It verifies that encryption occurs before file storage, passphrases are never stored or transmitted, and only authenticated users can access their own encrypted files. It also validates error handling for incorrect passphrases, invalid file inputs, and unauthorized access attempts across different browsers and devices.

Output Testing

Output testing focuses on verifying the correctness and usability of system outputs. This includes confirming that downloaded files exactly match the original uploaded files after decryption, validating correct file names and formats, and ensuring error messages are displayed clearly when decryption fails. The responsiveness and clarity of file lists, alerts, and download actions are also tested to ensure consistent behavior across various screen sizes and platforms.

CONCLUSION

CONCLUSION

The Secure File Storage Portal is designed with a security-focused and modular architecture to provide a reliable solution for protecting sensitive digital files using client-side encryption. By implementing AES-256-GCM encryption through the Web Crypto API and deriving keys from user-defined passphrases, the system ensures that files are encrypted before leaving the user's device. This zero-knowledge approach guarantees that neither the storage provider nor any intermediary can access the original data, thereby maintaining complete user privacy and control.

The architecture clearly separates authentication, encryption, storage management, and user interface layers to enhance maintainability and scalability. Users interact with a simple and intuitive dashboard that supports secure login, file upload, encrypted storage, and controlled file retrieval. The encryption and decryption processes are performed entirely within the browser, ensuring that plaintext files and encryption keys are never transmitted or stored externally. Metadata such as file names, timestamps, initialization vectors, and salts are managed securely to support accurate file recovery without compromising security.

Overall, the Secure File Storage Portal successfully demonstrates a practical implementation of client-side encryption for secure cloud storage. By combining modern web technologies with strong cryptographic principles and cloud hosting, the system delivers a secure, accessible, and efficient platform for confidential file management. This approach empowers users to store and retrieve sensitive data with confidence, even in untrusted cloud environments, while maintaining high usability and performance across devices.

BIBLIOGRAPHY

BIBLIOGRAPHY

BOOK REFERENCES

1. William Stallings, *Cryptography and Network Security: Principles and Practice*, 7th Edition, Pearson Education, 2017.

2. Jonathan Katz and Yehuda Lindell, *Introduction to Modern Cryptography*, 2nd Edition, CRC Press, 2014.

3. Adam Freeman, *Pro JavaScript Development*, A press, 2016.

4. Kyle Simpson, *You Don't Know JS Yet: Scope & Closures*, 2nd Edition, Independently Published, 2020.

5. Martin Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.

REFERENCE WEBSITE

1. https://developer.mozilla.org — MDN Web Docs for Web Crypto API, JavaScript, HTML, and CSS references

2. https://firebase.google.com — Firebase Authentication, Fire store, and Hosting documentation

3. https://vitejs.dev — Vite official documentation for modern frontend build tooling

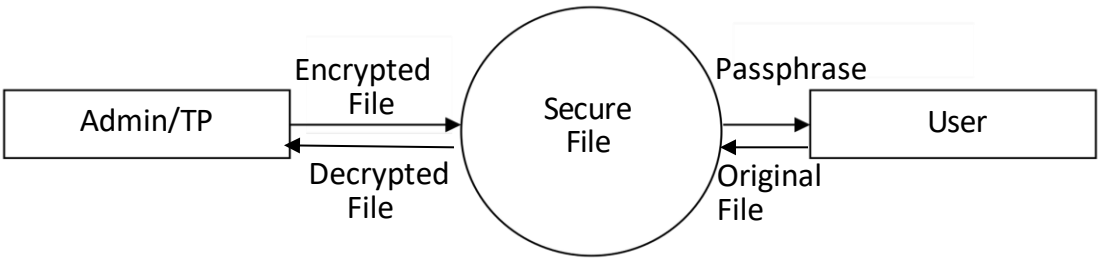
4. https://web.dev — Web security best practices and performance optimization guides

5. https://owasp.org — OWASP documentation for secure web application design and data protection

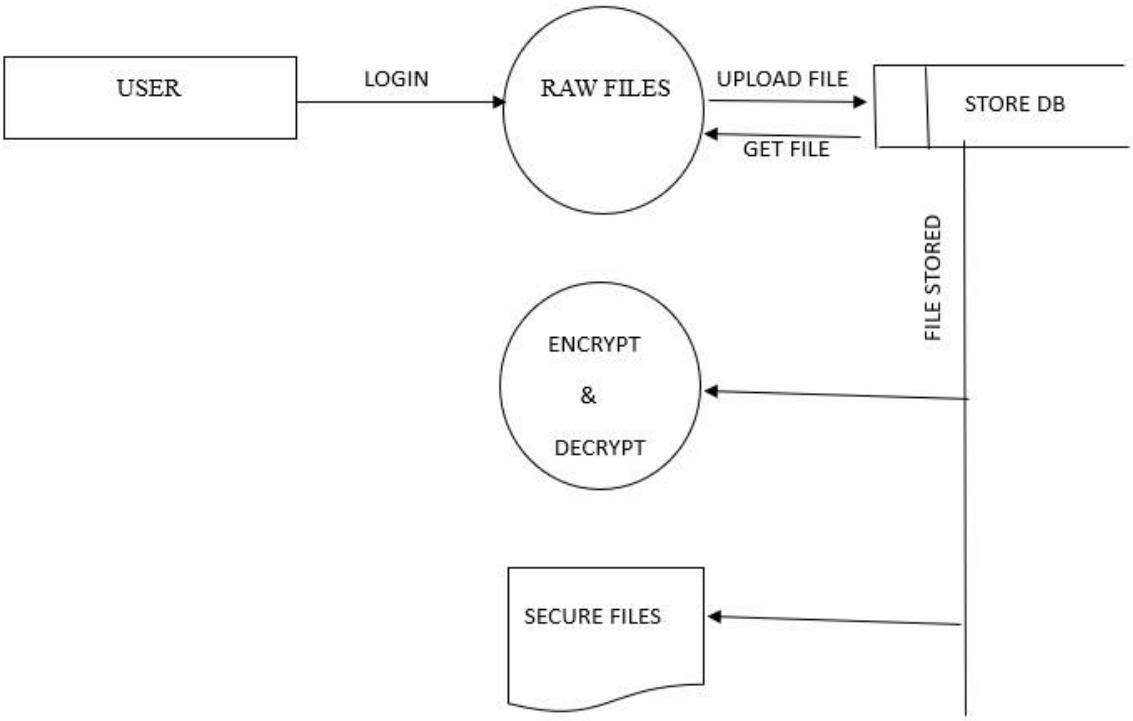
APPENDICES

A. DATA FLOW DIAGRAM

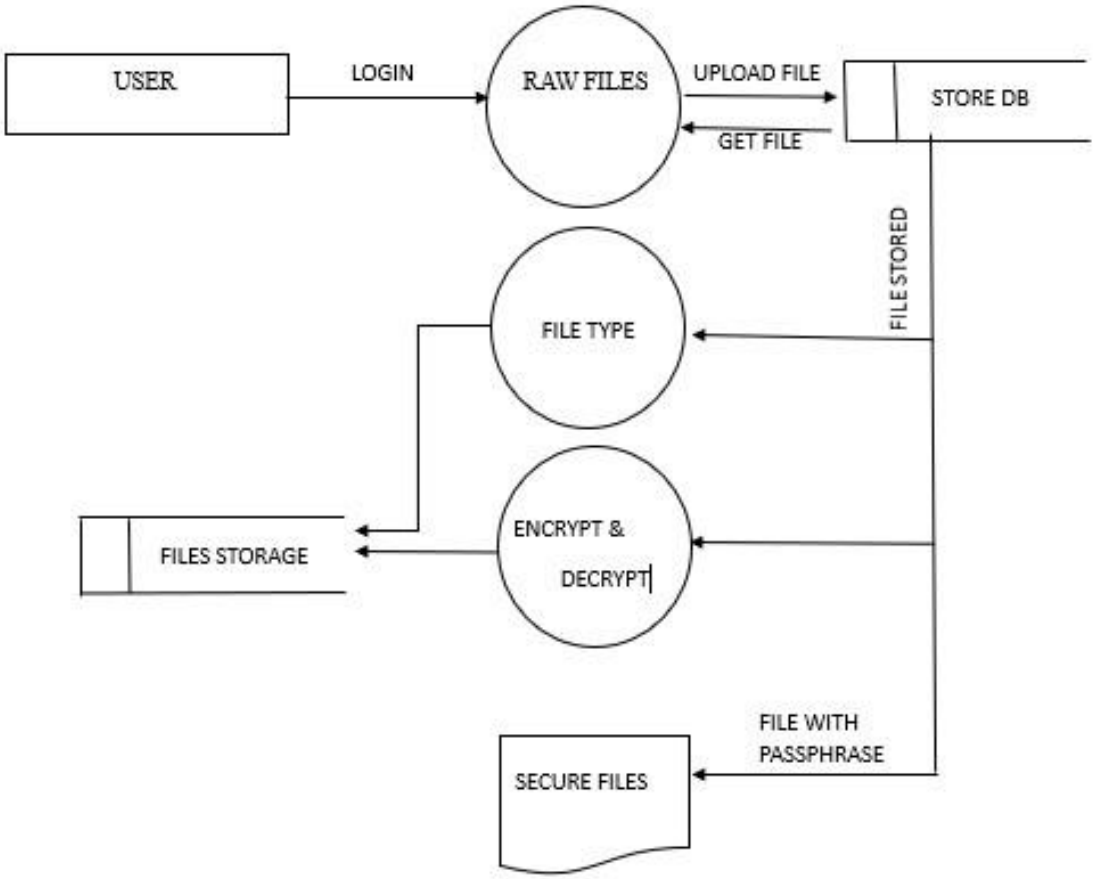
Level 0



Level 1



Level 2


USER
LOGIN
RAW FILES
UPLOAD FILE
GET FILE
STORE DB
FILE STORED
FILE TYPE
ENCRYPT &
DECRYPT
FILES STORAGE
FILE WITH
PASSPHRASE
SECURE FILES

B. TABLE STRUCTURE

Table name:

user design

Tracks user access sessions within the encryption system.

COLUMN NAME	DATA TYPE	DESCRIPTION
USER_ID	INT (PK)	Unique identifier for the user
SESSION_ID	VARCHAR	Unique identifier for a specific user session.
ACCESS_TIMESTAMP	VARCHAR	The exact date and time the user accessed the system.

Table name:

Encrypted file

Stores information about the files uploaded and secured by the user.

COLUMN NAME DATA TYPE DESCRIPTION

FILE_ID INT (PK) Unique identifier for the file.

USER_ID VARCHAR Links the file to the user who uploaded it.

FILE_NAME VARCHAR The original name of the uploaded file.

ENCRYPTED_DATA BLOB The actual binary content of the file after encryption.

UPLOAD_TIMESTAMP TIMESTAMP The date and time when the file was uploaded.

FILE_SIZE INT The size of the file in bytes.

Table name:

Encryption meta data

Technical parameters required to decrypt the associated files.

COLUMN NAME	DATA TYPE	DESCRIPTION
METADATA_ID	INT (PK)	Unique identifier for the metadata record.
FILE_ID	INT (FK)	Links the metadata to a specific encrypted file.
IV	INT (FK)	Initialization Vector used for the encryption process.
SALT	VARCHAR	PBKDF2 Salt used for key derivation.
ENCRYPTION_A LGOR ITHM	VARCHAR	The algorithm used (e.g., AES-256-GCM).
KEYDERIVATIO N_ME THOD	VARCHAR	The method used to derive the encryption key (e.g., PBKDF2).

C. SAMPLE CODING

```
import { encryptFile, decryptFile } from "./js/crypto.js";

import { saveFile, getAllFiles, getFile, deleteFile } from "./js/storage.js";

// Simple "auth" simulation (for demo purposes)
// In real, use Firebase Auth or proper backend
let currentUser = null;

// Elements
const authSection = document.getElementById("auth-section");
const dashboard = document.getElementById("dashboard");
const emailInput = document.getElementById("email");
const passwordInput = document.getElementById("password");
const loginBtn = document.getElementById("loginBtn");
const registerBtn = document.getElementById("registerBtn");
const logoutBtn = document.getElementById("logoutBtn");
const fileInput = document.getElementById("fileInput");
const passphraseInput = document.getElementById("passphrase");
const uploadBtn = document.getElementById("uploadBtn");
const fileList = document.getElementById("fileList");

// Simple localStorage "users" (demo only, NOT secure!) function getUsers() {
return JSON.parse(localStorage.getItem("users") || "{}"); }
function saveUsers(users) {
localStorage.setItem("users", JSON.stringify(users));
}

function login(email, password) {
const users = getUsers();
if (users[email] && users[email] === password) {
currentUser = email;
return true;
}
return false;
}

function register(email, password) {
const users = getUsers();
if (users[email]) return false; // user exists
users[email] = password;
saveUsers(users);
return true;
} function logout() {
currentUser = null;
showAuth();
}
```

```
function showAuth() {
authSection.style.display = "block";
dashboard.style.display = "none";
clearInputs();
}

function showDashboard() {
authSection.style.display = "none";
dashboard.style.display = "block";
renderFileList();
clearInputs();
}

function clearInputs() {
emailInput.value = "";
passwordInput.value = "";
passphraseInput.value = "";
fileInput.value = "";
}

// Event Listeners
loginBtn.addEventListener("click", () => {
const email = emailInput.value.trim();
const password = passwordInput.value.trim();
if (!email || !password) return alert("Enter email & password");
if (login(email, password)) { alert("Logged in!");
showDashboard();
} else {
alert("Invalid credentials");
}
});

registerBtn.addEventListener("click", () => {
const email = emailInput.value.trim();
const password = passwordInput.value.trim();
if (!email !password) return alert("Enter email & password");
if (register(email, password)) { alert("Registered! Please login.");
} else {
alert("User already exists.");
}
});

logoutBtn.addEventListener("click", () => {
logout();

alert("Logged out.");
});

uploadBtn.addEventListener("click", async () =>
```

```
{

 const file = fileInput.files[0]; const passphrase = passphraseInput.value.trim();

// Show/hide sections
if (!file || !passphrase) {
alert(" Please select a file and enter a passphrase"); return;
}

uploadBtn.disabled = true;
uploadBtn.textContent = Encrypting...";

try {
// Encrypt file
const { encryptedBlob, iv, salt } = await encryptFile(file, passphrase);

// Store user-specific data
const id = crypto.randomUUID();
const storedFile = {
id,
owner: currentUser,
fileName: file.name,
iv,
salt,
encryptedData: encryptedBlob,
createdAt: new Date().toISOString(),
};

await saveFile(storedFile);
alert("File encrypted and saved securely! );
renderFileList();

// Clear inputs after upload
passphraseInput.value = "";
fileInput.value = "";
} catch (error) {
alert("Encryption failed: " error.message);
} finally {
uploadBtn.disabled = false;
uploadBtn.textContent = Upload & Encrypt";
}
});

async function renderFileList() {
const allFiles = await getAllFiles();
const userFiles = allFiles.filter((f) => f.owner === currentUser);
fileList.innerHTML = "";

if (userFiles.length === 0) {
fileList.innerHTML = "<p>No files stored.</p>";
```

```
return;
}

userFiles.forEach((file) => {
const li = document.createElement("li");
fileInfo.className = "file-info";
fileInfo.innerHTML=`<strong>${file.fileName}</strong><br><small>Saved:
${new Date(file.createdAt).toLocaleString()}</small>`;
li.appendChild(fileInfo);
const fileActions = document.createElement("div");
fileActions.className = "file-actions";

const downloadBtn = document.createElement("button");
downloadBtn.className = "btn btn-success";
downloadBtn.textContent = "Download & Decrypt";
downloadBtn.onclick = async () => {
const passphrase = prompt(" Enter passphrase to decrypt this file:");
if (!passphrase) return alert(" Passphrase required");

try {
const fileRecord = await getFile(file.id);
const decryptedBlob = await decryptFile(fileRecord.encryptedData, passphrase, fileRecord.iv,
fileRecord.salt);

// Download restored file
const url = URL.createObjectURL(decryptedBlob);
const a = document.createElement("a");
a.href = url;
a.download = fileRecord.fileName;
document.body.appendChild(a);
a.click();
a.remove();
URL.revokeObjectURL(url);
a l e rt ( "File decrypted and downloaded successfully!");
} catch (e) {
alert("Decryption failed. Incorrect passphrase or corrupted file.");
}
};
 const encryptBtn = document.createElement("button");
encryptBtn.className = "btn btn-secondary";
encryptBtn.textContent = Download Encrypted";
encryptBtn.onclick = async () => {
try {
const fileRecord = await getFile(file.id);
const url = URL.createObjectURL(fileRecord.encryptedData);
const a = document.createElement("a");

a.href = url;
a.download = `${fileRecord.fileName}.encrypted`;
document.body.appendChild(a);
a.click();
```

```
a.remove();
alert("Encrypted file downloaded successfully!");
URL.revokeObjectURL(url);

} catch (e) {
alert("Failed to download encrypted file.");
}
const fileInfo = document.createElement("div");
};
const deleteBtn = document.createElement("button");
deleteBtn.className = "btn btn-danger";
deleteBtn.textContent = "Delete";
deleteBtn.onclick = async () => {
if (confirm("Are you sure you want to delete this file?"))
{ await deleteFile(file.id); renderFileList();
a l e rt( "File deleted successfully!");
}
};

fileActions.appendChild(downloadBtn);
fileActions.appendChild(encryptBtn);
fileActions.appendChild(deleteBtn);
li.appendChild(fileActions);
fileList.appendChild(li);
});
}

// Show auth form initially
showAuth();
const DB_NAME = "SecureFileStorageDB";
const STORE_NAME = "encryptedFiles";

function openDB() {
return new Promise((resolve, reject) => {
const request = indexedDB.open(DB_NAME, 1);

request.onerror = () => reject(request.error);
request.onsuccess = () => resolve(request.result);

request.onupgradeneeded = (e) => {
const db = e.target.result;
if (!db.objectStoreNames.contains(STORE_NAME))
{ db.createObjectStore(STORE_NAME, { keyPath: "id" }); }
};
});
}

export async function saveFile(fileObj) {
const db = await openDB();
return new Promise((resolve, reject) => {
const tx = db.transaction(STORE_NAME, "readwrite");
```

```
const store = tx.objectStore(STORE_NAME);
const request = store.add(fileObj);

request.onsuccess = () => resolve(true);
request.onerror = () => reject(request.error);
});
}
export async function getFile(id) {
const db = await openDB();
return new Promise((resolve, reject) => {
const tx = db.transaction(STORE_NAME, "readonly");
const store = tx.objectStore(STORE_NAME);
const request = store.get(id);

request.onsuccess = () => resolve(request.result);

request.onerror = () => reject(request.error);
});
}

export async function getAllFiles() {
const db = await openDB();
return new Promise((resolve, reject) => {
const tx = db.transaction(STORE_NAME, "readonly");

const store = tx.objectStore(STORE_NAME);
const request = store.getAll();

request.onsuccess = () => resolve(request.result);

request.onerror = () => reject(request.error);  });
}

export async function deleteFile(id) {
const db = await openDB();
return new Promise((resolve, reject) => {
const tx = db.transaction(STORE_NAME, "readwrite");
const store = tx.objectStore(STORE_NAME);
const request = store.delete(id);

request.onsuccess = () => resolve(true);
request.onerror = () => reject(request.error);
});
}

// crypto.js — Client-side encryption utilities

const encoder = new TextEncoder();
const decoder = new TextDecoder();
```

```
/**
* Derive encryption key from passphrase
*/
async function deriveKey(passphrase, salt) {
const baseKey = await crypto.subtle.importKey( "raw",
encoder.encode(passphrase),
"PBKDF2",
false,
["deriveKey"]
);
return crypto.subtle.deriveKey(
{
name: "PBKDF2",
salt,
iterations: 100000,
hash: "SHA-256"
},
baseKey,
{
name: "AES-GCM",
length: 256
},
false,
["encrypt", "decrypt"]
);
}

/**
* Encrypt file
*/
export async function encryptFile(file, passphrase) {
const iv = crypto.getRandomValues(new Uint8Array(12));
const salt = crypto.getRandomValues(new Uint8Array(16));
const key = await deriveKey(passphrase, salt);

const fileBuffer = await file.arrayBuffer();

const encrypted = await crypto.subtle.encrypt(
{ name: "AES-GCM", iv },
key,
fileBuffer
);
return {
encryptedBlob: new Blob([encrypted]),
iv: Array.from(iv),
salt: Array.from(salt)
};
}
```

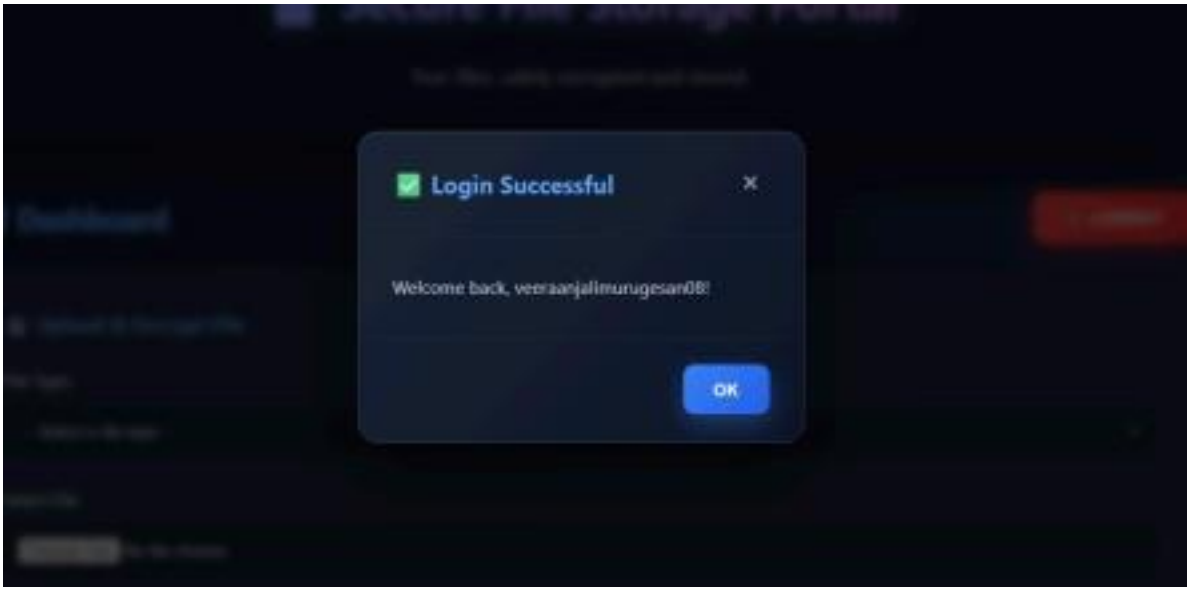
```
/**
* Decrypt file
*/
export async function decryptFile(encryptedBlob, passphrase, iv, salt)
{ const key = await deriveKey(
passphrase,
new Uint8Array(salt)
);
const encryptedBuffer = await encryptedBlob.arrayBuffer();

const decrypted = await crypto.subtle.decrypt(
{ name: "AES-GCM", iv: new Uint8Array(iv) },
key,
encryptedBuffer
);
return new Blob([decrypted]);
} import { encryptFile } from "./crypto.js";
import { saveFile } from "./storage.js";
/**
* Encrypt and upload a file (save to IndexedDB)
* @param {File} file - The File object selected by user
* @param {string} passphrase - The user provided encryption passphrase
* @param {string} owner - The current logged in user/email/id
* @returns {Promise<void>} */
export async function encryptAndUpload(file, passphrase, owner) {
if (!file) throw new Error("No file provided");
if (!passphrase) throw new Error("Passphrase required");
if (!owner) throw new Error("Owner info missing");
// Encrypt file using your crypto module
const { encryptedBlob, iv, salt } = await encryptFile(file, passphrase);
// Create a unique id for this file
const id = crypto.randomUUID();
// Prepare the record object with metadata
const fileRecord = { id, owner, fileName: file.name, iv, salt, encryptedData: encryptedBlob,
createdAt: new Date().toISOString(), };

// Save encrypted file + metadata to IndexedDB
await saveFile(fileRecord);
}
```

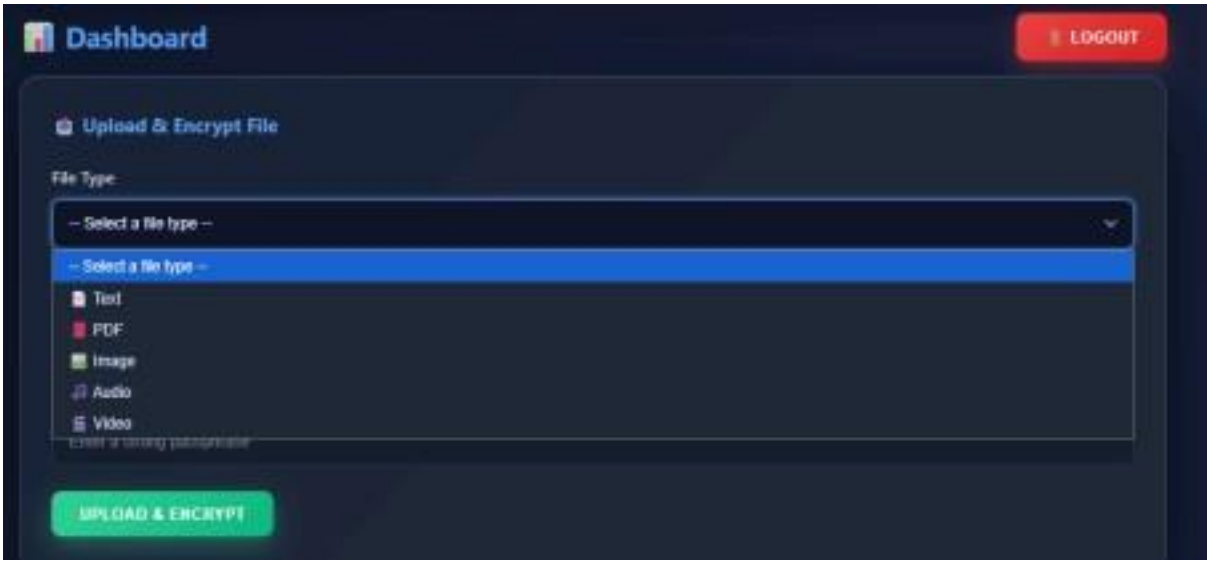
D. SAMPLE INPUT

LOGIN PAGE



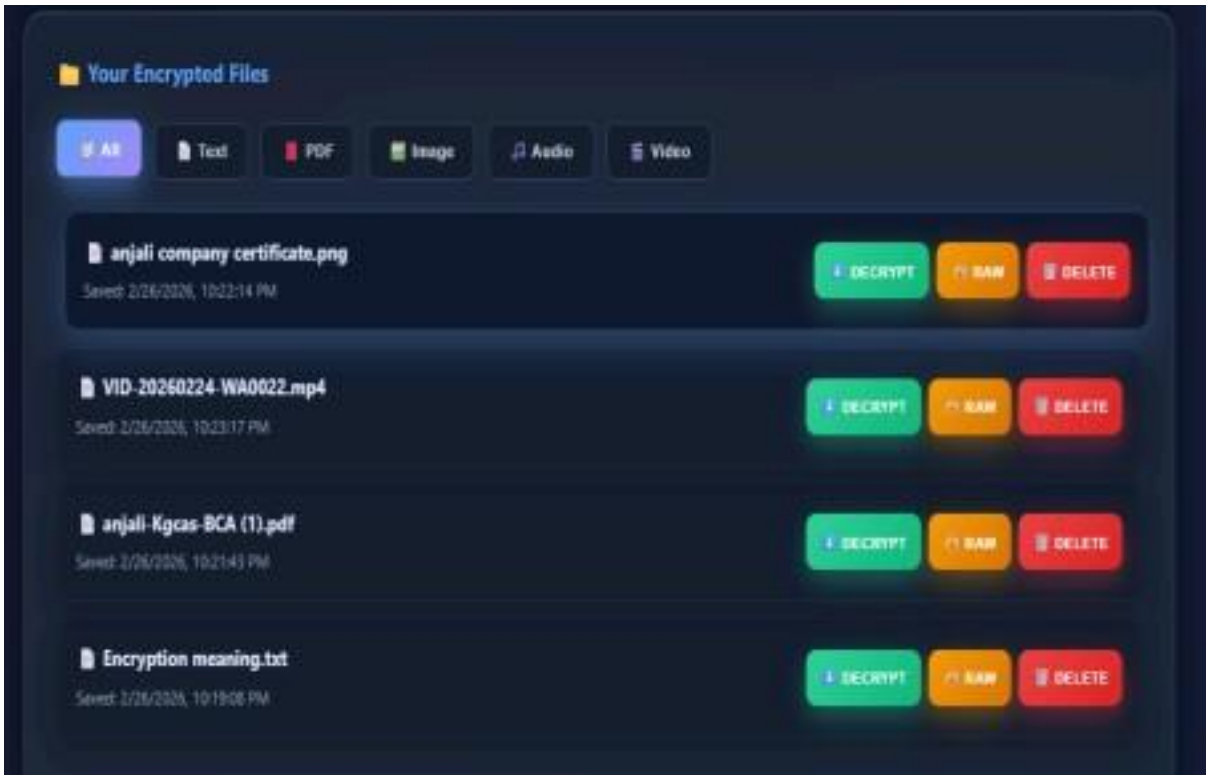
LOGIN SUCCESSFUL

DASHBOARD



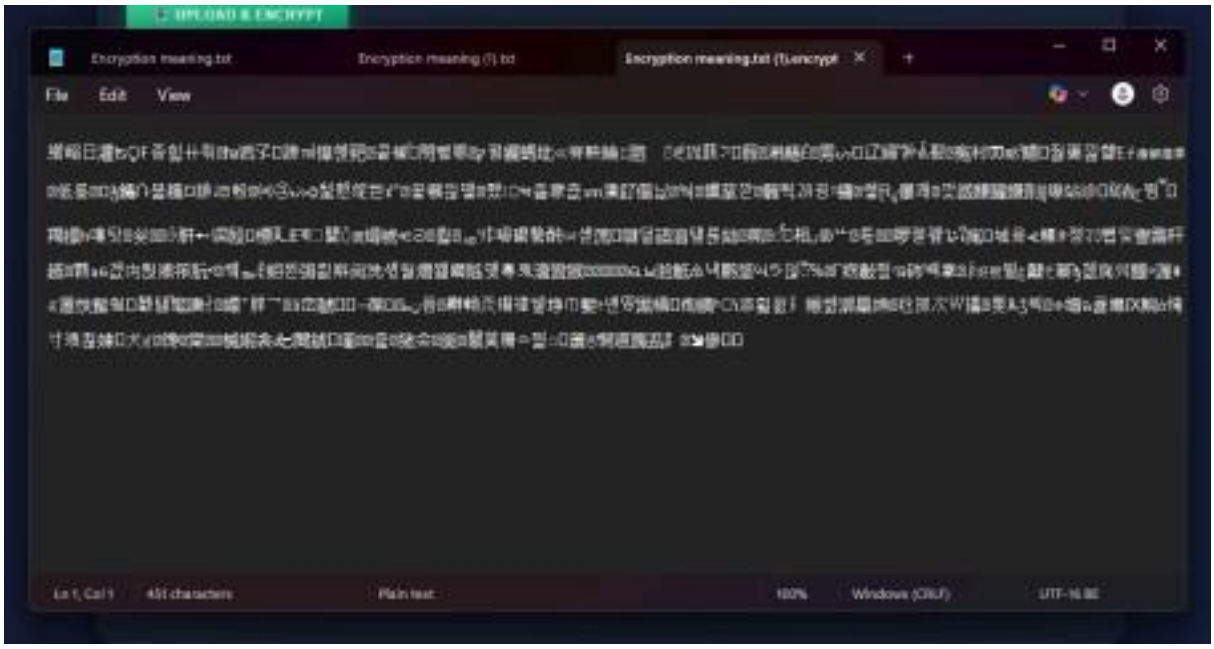
FILE TYPE

FILE UPLOADED

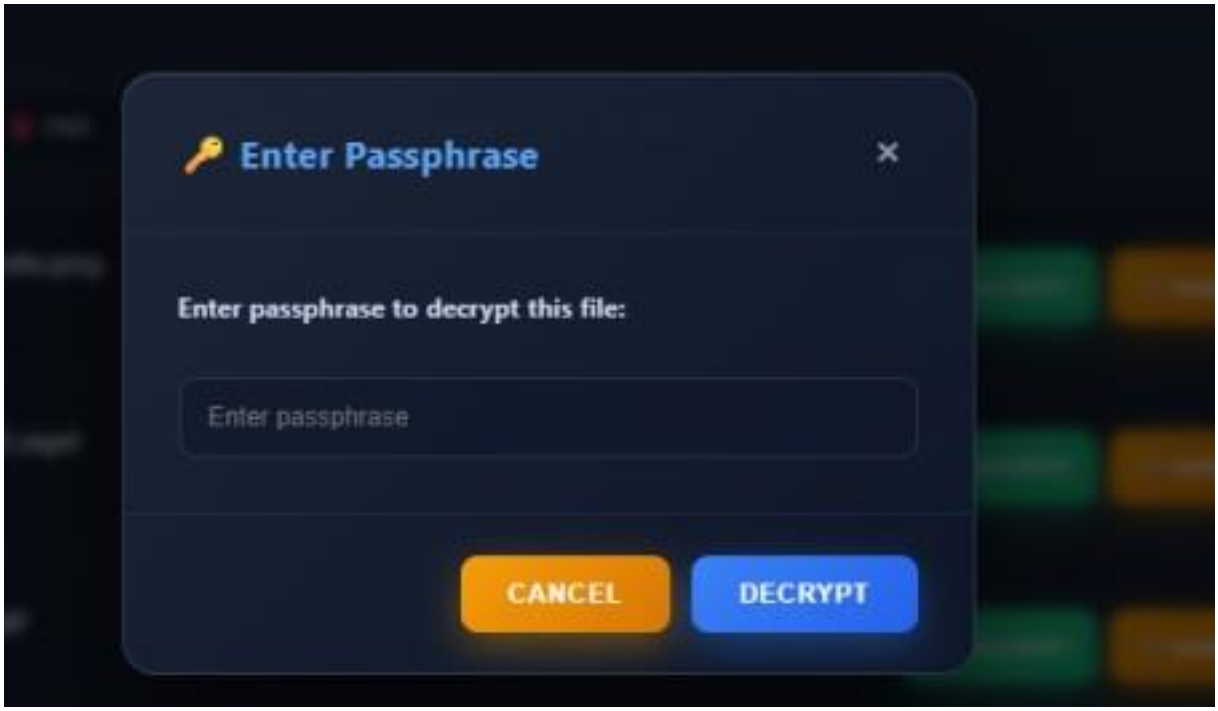


LIST OF FILES

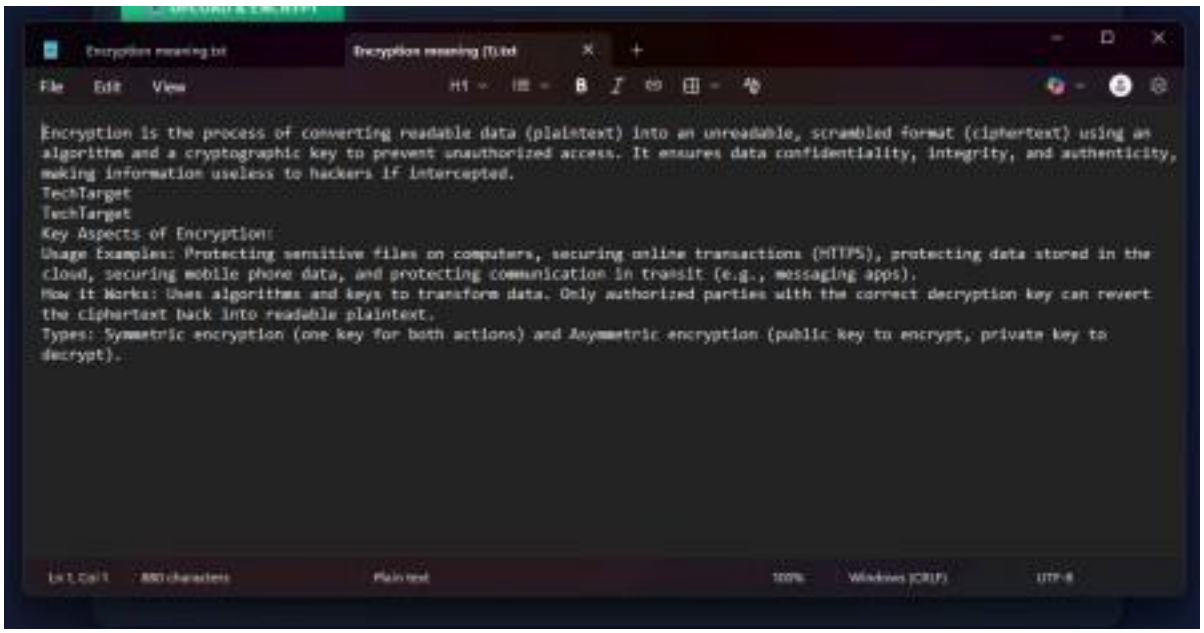
E. SAMPLE OUTPUT



ENCRYPTED FORMAT



DECRYPTED VERIFICATION



DECRYPTED FILE